

Установка и настройка Visual Studio Code

Установка Visual Studio Code

1. Загрузить установщик по ссылке: <https://go.microsoft.com/fwlink/?LinkID=534107>
2. После загрузки запустите установщик (VSCodeUserSetup-{версия}.exe).
3. По умолчанию VS Code установлен в
C:\Users\{Username}\AppData\Local\Programs\Microsoft VS Code.

Настройка для работы с C++

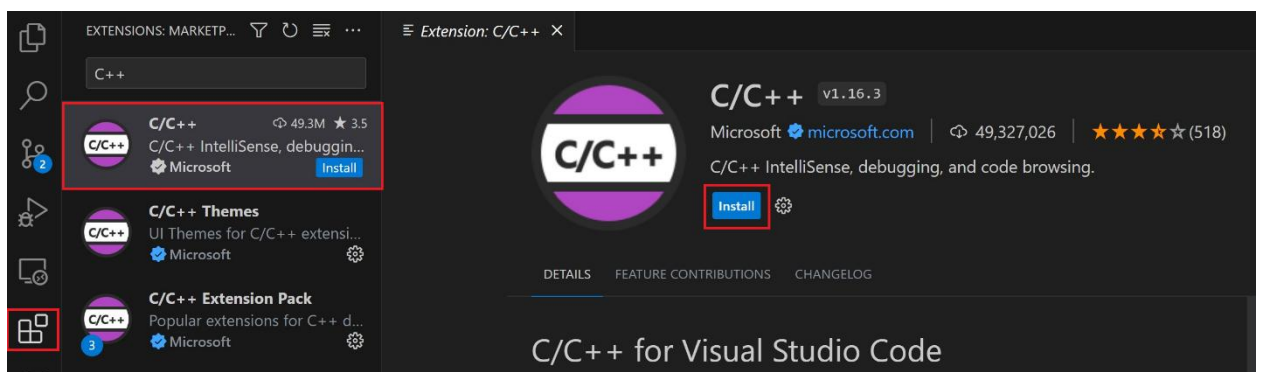
Использование GCC с MinGW

В этом руководстве вы настроите Visual Studio Code для использования компилятора GCC C++ (g++) и отладчика GDB из [mingw-w64](#) для создания программ, которые будут работать в Windows. После настройки VS Code вы скомпилируете, запустите и отладите программу Hello World.

Предварительные условия

Чтобы успешно завершить это руководство, вы должны выполнить следующие шаги:

1. Установите [Visual Studio Code](#).
2. Установите [расширение C/C++ для VS Code](#). Вы можете установить расширение C/C++, выполнив поиск «C++» в разделе «Расширения» (Ctrl+Shift+X).



Получите последнюю версию MinGW-w64 через [MSYS2](#), которая предоставляет актуальные нативные сборки GCC, MinGW-w64 и других полезных инструментов и библиотек C++. Это предоставит вам необходимые инструменты для компиляции кода, его отладки и настройки для работы с [IntelliSense](#).

1. Вы можете скачать последнюю версию установщика со страницы MSYS2 или воспользоваться этой [прямой ссылкой на установщик](#).
2. Запустите установщик и следуйте инструкциям мастера установки. Обратите внимание, что для MSYS2 требуется 64-разрядная версия Windows 8.1 или более новая.
3. В мастере установки выберите нужную папку для установки. Запишите этот каталог на будущее. В большинстве случаев рекомендуемый каталог подходит. То же самое относится к шагу настройки ярлыков в меню «Пуск». После завершения убедитесь, что установлен флажок Запустить MSYS2 сейчас, и выберите Готово. Откроется окно терминала MSYS2.
4. В этом терминале установите набор инструментов MinGW-w64, выполнив следующую команду:

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

5. Примите количество пакетов в toolchain группе по умолчанию, нажав Enter.

```
$ pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
warning: base-devel-2022.12-2 is up to date -- skipping
:: There are 19 members in group mingw-w64-ucrt-x86_64-toolchain:
:: Repository ucrt64
 1) mingw-w64-ucrt-x86_64-binutils  2) mingw-w64-ucrt-x86_64-crt-git
 3) mingw-w64-ucrt-x86_64-gcc      4) mingw-w64-ucrt-x86_64-gcc-ada
 5) mingw-w64-ucrt-x86_64-gcc-fortran 6) mingw-w64-ucrt-x86_64-gcc-libgfortran
 7) mingw-w64-ucrt-x86_64-gcc-libs  8) mingw-w64-ucrt-x86_64-gcc-objc
 9) mingw-w64-ucrt-x86_64-gdb      10) mingw-w64-ucrt-x86_64-gdb-multiarch
11) mingw-w64-ucrt-x86_64-headers-git 12) mingw-w64-ucrt-x86_64-libgccjit
13) mingw-w64-ucrt-x86_64-libmangle-git 14) mingw-w64-ucrt-x86_64-libwinpthread-git
15) mingw-w64-ucrt-x86_64-make     16) mingw-w64-ucrt-x86_64-pkgconf
17) mingw-w64-ucrt-x86_64-tools-git 18) mingw-w64-ucrt-x86_64-winpthreads-git
19) mingw-w64-ucrt-x86_64-winstorecompat-git

Enter a selection (default=all): |
```

6. Введите Y, когда будет предложено продолжить установку.
7. Установите сборщик make выполнив следующую команду:

```
pacman -S make
```

Добавьте путь к папке MinGW-w64 bin в переменную среды Windows PATH с помощью следующих шагов:

1. В строке поиска Windows введите **«Настройки»**, чтобы открыть «Настройки Windows».
2. Выполните поиск для **редактирования переменных среды для вашей учетной записи**.
3. В разделе **«Пользовательские переменные»** выберите переменную Path и нажмите **«Изменить»**.
4. Выберите **«Создать»** и добавьте в список папку назначения MinGW-w64, которую вы записали в процессе установки. Если вы использовали приведенные выше настройки по умолчанию, то это будет путь: C:\msys64\ucrt64\bin и C:\msys64\usr\bin.
5. Нажмите **ОК**, а затем снова нажмите **ОК** в окне **Переменные среды**, чтобы обновить PATH переменную среды. Вам необходимо повторно открыть все окна консоли, чтобы обновленная PATH переменная окружения была доступна.

Проверка установки

Чтобы проверить, правильно ли установлены и доступны ли ваши инструменты MinGW-w64, откройте новую командную строку и введите:

```
gcc --version
g++ --version
gdb -version
```

Вы должны увидеть вывод, в котором указано, какие версии GCC, g++ и GDB у вас установлены. Если это не так:

1. Убедитесь, что запись в переменной PATH соответствует двоичному расположению MinGW-w64, где была установлена цепочка инструментов. Если компиляторы не найдены в этой записи PATH, убедитесь, что вы следовали предыдущим инструкциям.
2. Если gcc выдаёт правильный результат, но не gdb, то вам нужно установить недостающие пакеты из набора инструментов MinGW-w64.
 - Если при компиляции вы получаете сообщение «Значение miDebuggerPath недопустимо», одной из причин может быть отсутствие пакета mingw-w64-gdb.

Создание приложения Hello Word

Настроим проект:

1. Запустите командную строку Windows (введите командную строку Windows в строке поиска Windows).

2. Выполните следующие команды. Они создадут пустую папку с именем projects, куда вы сможете поместить все свои проекты VS Code. Там следующие команды создадут подпапку с именем helloworld и перейдут к ней. Оттуда вы откроете helloworld непосредственно в VS Code.

```
mkdir projects
cd projects
mkdir helloworld
cd helloworld
code .
```

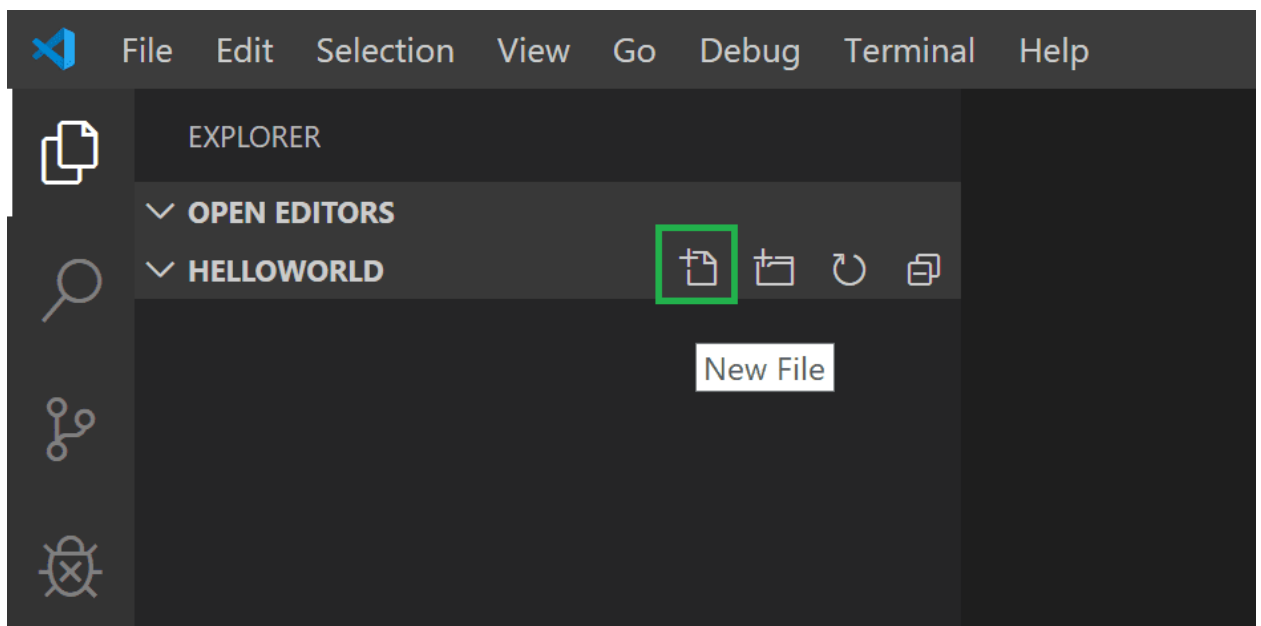
Команда «code .» открывает VS Code в текущей рабочей папке, которая становится вашей «рабочей областью». Примите диалоговое окно [«Доверие к рабочей области»](#), выбрав **«Да, я доверяю авторам»**, поскольку это папка, которую вы создали.

По мере прохождения урока вы увидите три файла, созданные в папке `.vscode` в рабочей области:

- `tasks.json` (инструкции по сборке)
- `launch.json` (настройки отладчика)
- `c_cpp_properties.json` (путь к компилятору и настройки IntelliSense)

Добавление файла исходного кода Hello World

В строке заголовка проводника выберите кнопку **«Новый файл»** и назовите файл `helloworld.cpp`.



Добавьте исходный код hello world

Вставьте в этот исходный код:

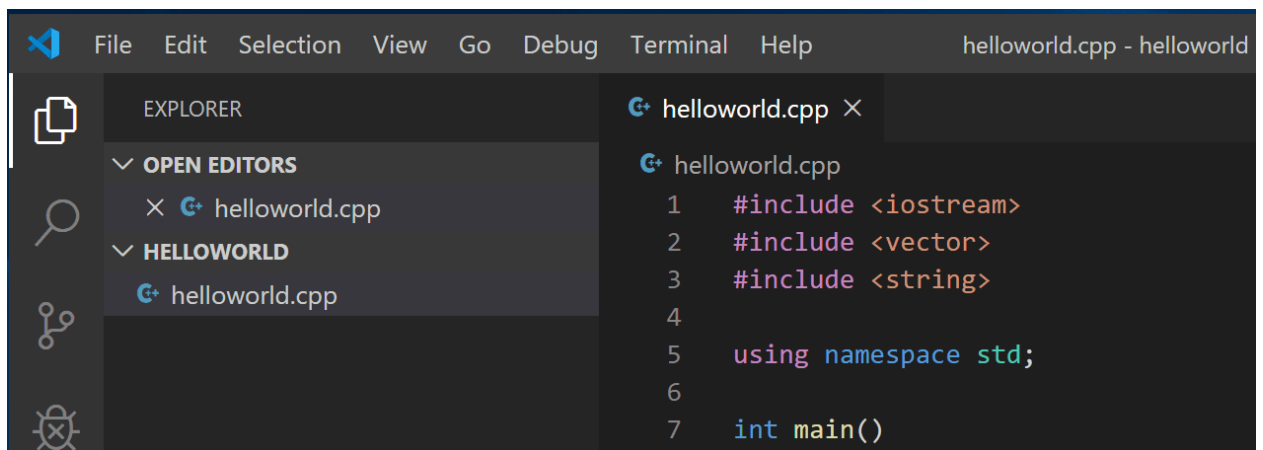
```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

int main()
{
    vector<string> msg {"Hello", "C++", "World", "from", "VS Code", "and the
C++ extension!"};

    for (const string& word : msg)
    {
        cout << word << " ";
    }
    cout << endl;
}
```

Теперь нажмите Ctrl+S, чтобы сохранить файл. Обратите внимание, что только что добавленный файл отображается в представлении **Проводника** (Ctrl+Shift+E) на боковой панели VS Code:

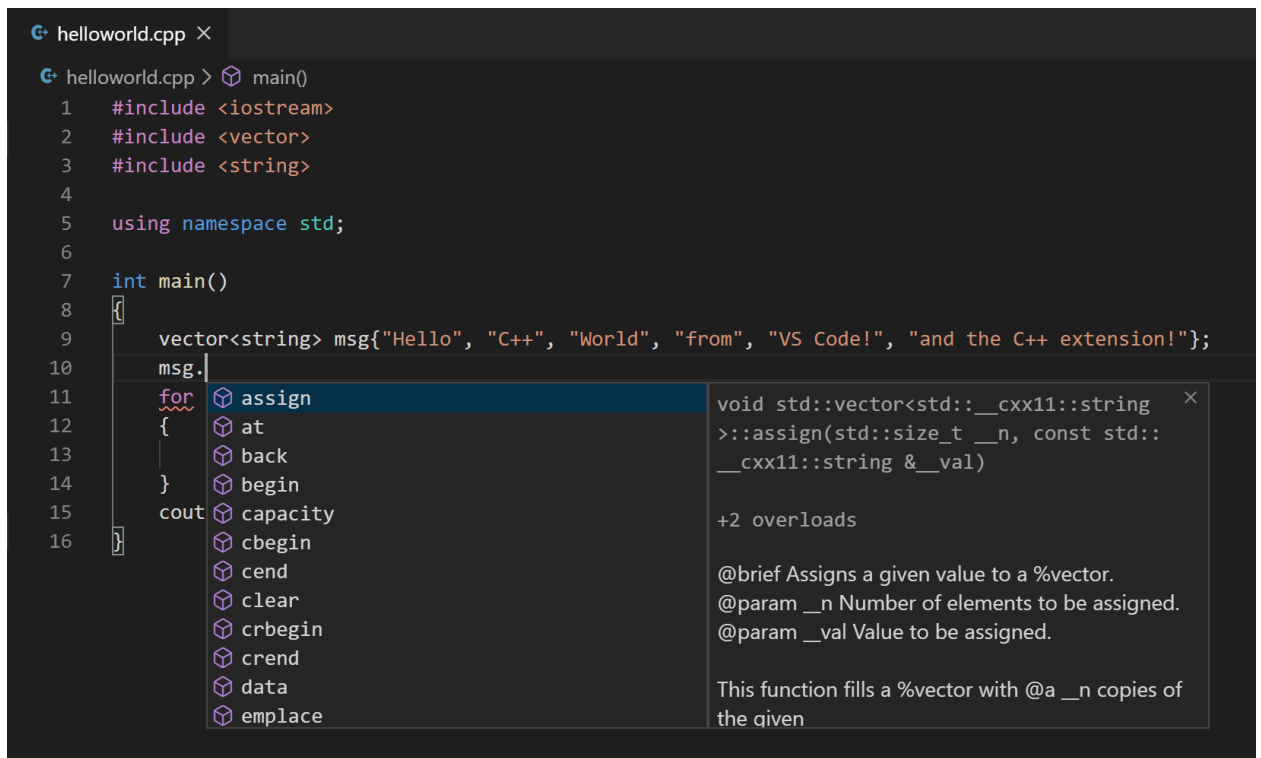


Вы также можете включить [Автосохранение](#) для автоматического сохранения изменений в файле, выбрав **Файл > Автосохранение**.

Исследуйте IntelliSense

IntelliSense — это инструмент, который помогает писать код быстрее и эффективнее, добавляя функции редактирования кода, такие как автозаполнение кода, информация о параметрах, краткая информация и списки элементов.

Чтобы увидеть IntelliSense в действии, наведите курсор на `vector` или `string` и посмотрите информацию об их типе. Если вы введете `msg.` в строке 10, вы увидите список рекомендуемых функций-членов для вызова, сгенерированный IntelliSense:



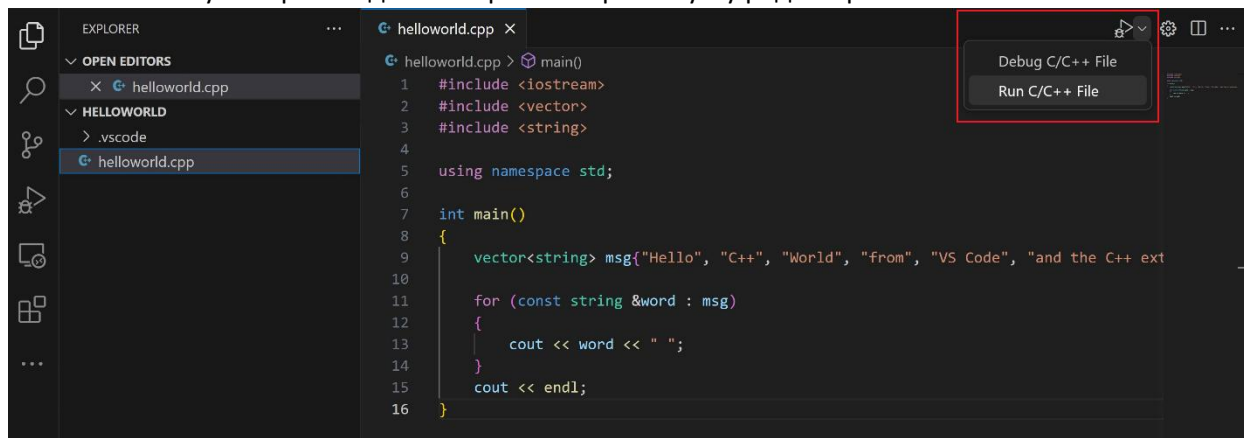
Вы можете нажать клавишу Tab, чтобы вставить выбранный элемент. Если вы затем добавите открывающие скобки, IntelliSense покажет информацию о том, какие аргументы необходимы.

Если IntelliSense ещё не настроен, откройте командную палитру (Ctrl+Shift+P) и введите «**Выбрать конфигурацию IntelliSense**». В раскрывающемся списке компиляторов выберите Use gcc.exe для настройки.

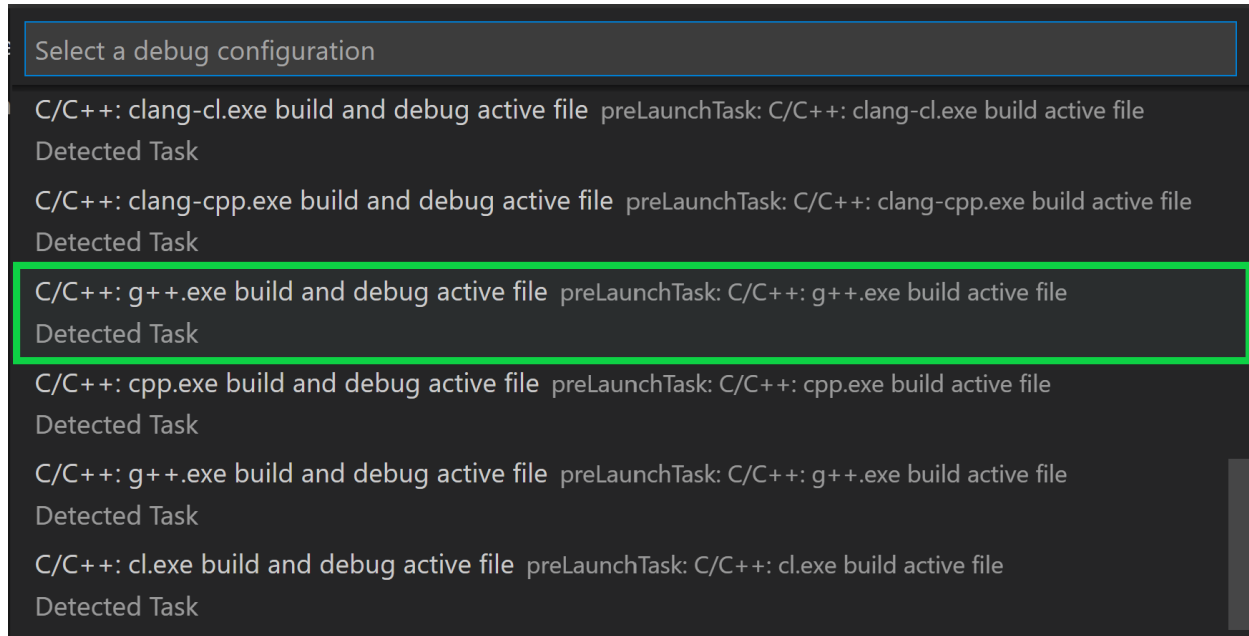
Компиляция и запуск helloworld.cpp

Расширение C++ использует компилятор C++, установленный на вашем компьютере, для создания вашей программы. Убедитесь, что вы выполнили шаг «Установка набора инструментов MinGW-w64» перед попыткой запуска и отладки helloworld.cpp в VS Code.

1. Открыть helloworld.cpp так чтобы он выбрался как активный файл.
2. Нажмите кнопку воспроизведения в правом верхнем углу редактора.

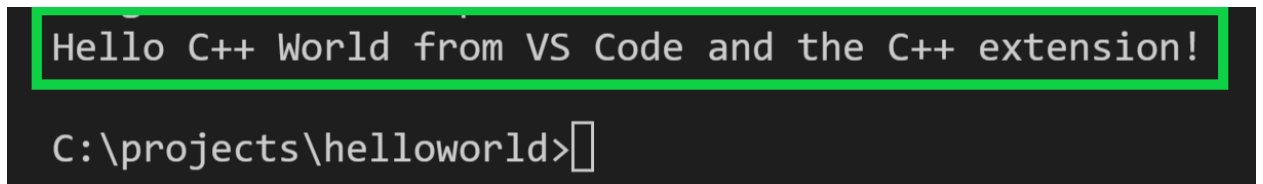


3. Выберите C/C++: g++.exe для сборки и отладки активного файла из списка обнаруженных компиляторов в вашей системе.



Вам будет предложено выбрать компилятор только при первом запуске `helloworld.cpp`. Этот компилятор будет установлен в качестве компилятора по умолчанию в `tasks.json` файле.

1. После успешной сборки выходные данные вашей программы появятся во встроенном **Терминале**.

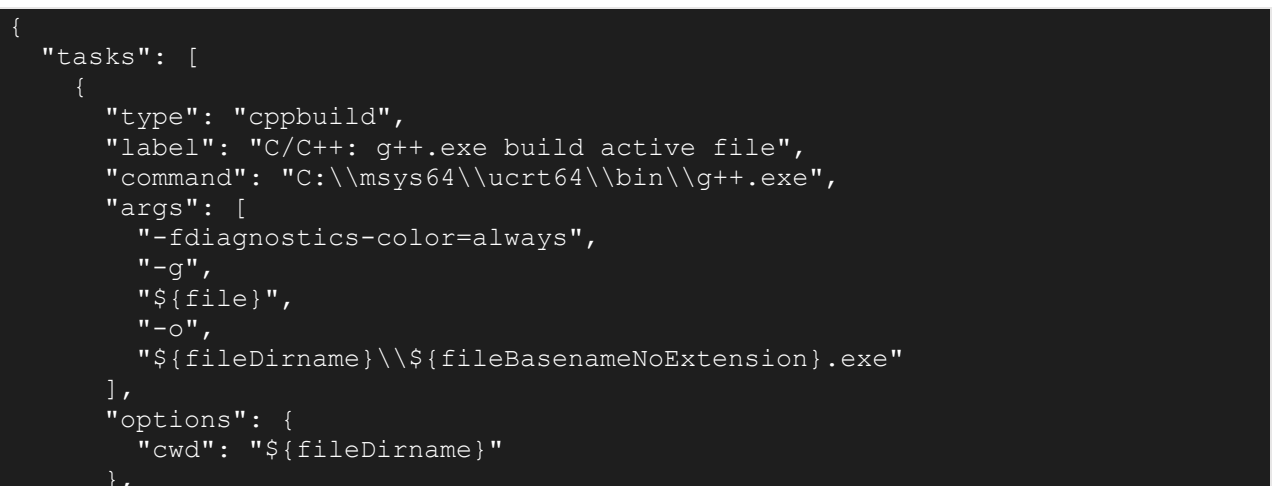


Вы только что запустили свою первую программу на C++ в VS Code

Понимание `tasks.json`

При первом запуске вашей программы расширение C++ создает `tasks.json` файл, который вы найдете в папке вашего проекта `.vscode` в разделе `tasks.json` r конфигурации сборки.

Ваш новый `tasks.json` файл должен выглядеть аналогично приведенному ниже JSON:



```

    "problemMatcher": ["$gcc"],
    "group": {
      "kind": "build",
      "isDefault": true
    },
    "detail": "Task generated by Debugger."
  },
  "version": "2.0.0"
}

```

Параметр `command` указывает на программу, которую нужно запустить; в данном случае это `g++`.

Массив `args` определяет аргументы командной строки, передаваемые в `g++`. Эти аргументы перечислены в этом файле в том порядке, который ожидает компилятор.

Эта задача говорит `g++` взять активный файл (`${file}`), скомпилировать его и создать выходной файл (`-o` с ключом) в текущем каталоге (`${fileDirname}`) с тем же именем, что и у активного файла, но с расширением `.exe` (`${fileBasenameNoExtension}.exe`). Для нас это приводит к `helloworld.exe`.

Значение `label` — это то, что вы увидите в списке задач; вы можете назвать его как угодно.

Значение `detail` — это то, что вы увидите в качестве описания задачи в списке задач. Настоятельно рекомендуется переименовать это значение, чтобы отличать его от похожих задач.

Значение `problemMatcher` определяет синтаксический анализатор, который будет использоваться для поиска ошибок и предупреждений в выводе компилятора. Для GCC наилучшие результаты будут получены при использовании `$gcc` средства поиска проблем.

С этого момента кнопка воспроизведения будет считывать данные из `tasks.json` для определения способа сборки и запуска вашей программы. Вы можете определить несколько задач сборки в `tasks.json`, и кнопка воспроизведения будет использовать ту задачу, которая отмечена как задача по умолчанию. Если вам нужно изменить компилятор по умолчанию, вы можете запустить **«Задачи: настроить задачу сборки по умолчанию»** в командной панели. Кроме того, вы можете изменить файл `tasks.json` и удалить задачу по умолчанию, заменив этот фрагмент:

```

"group": {
  "kind": "build",
  "isDefault": true
},

```

Изменение tasks.json

Начиная с 3 ноября 2024 года MSYS2 по умолчанию отключает поддержку подстановочных знаков для `mingw-w64`. Это изменение влияет на то, как подстановочные знаки, такие как `"*.cpp"`, обрабатываются в командах сборки. Чтобы собрать несколько файлов C++ в вашем `tasks.json`, вы должны явно указать файлы, использовать систему сборки, такую как `make` или `cmake`, или реализовать следующие обходные пути: <https://www.msys2.org/docs/c/#expanding-wildcard-arguments>.

Если вы ранее использовали `"${workspaceFolder}/*.cpp"` для компиляции всех `.cpp` файлов в текущей папке, это больше не будет работать напрямую. Вместо этого вы можете вручную перечислить файлы или определить сценарий сборки.

Отладка helloworld.cpp

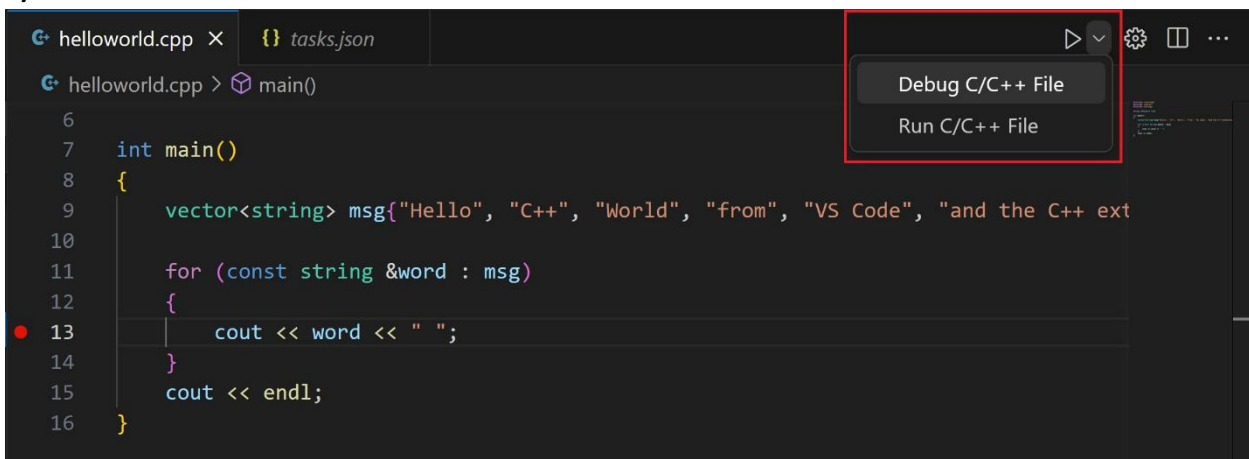
Для отладки вашего кода,

1. Вернитесь к `helloworld.cpp`, чтобы это был активный файл.
2. Установите точку останова, нажав на край редактора или нажав F9 на текущей строке.



```
helloworld.cpp X
helloworld.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      vector<string> msg{"Hello", "C++", "World", "from", "VS Code", "and the C++ extension!"};
10     for (const string &word : msg)
11     {
12         cout << word << " ";
13     }
14     cout << endl;
15 }
```

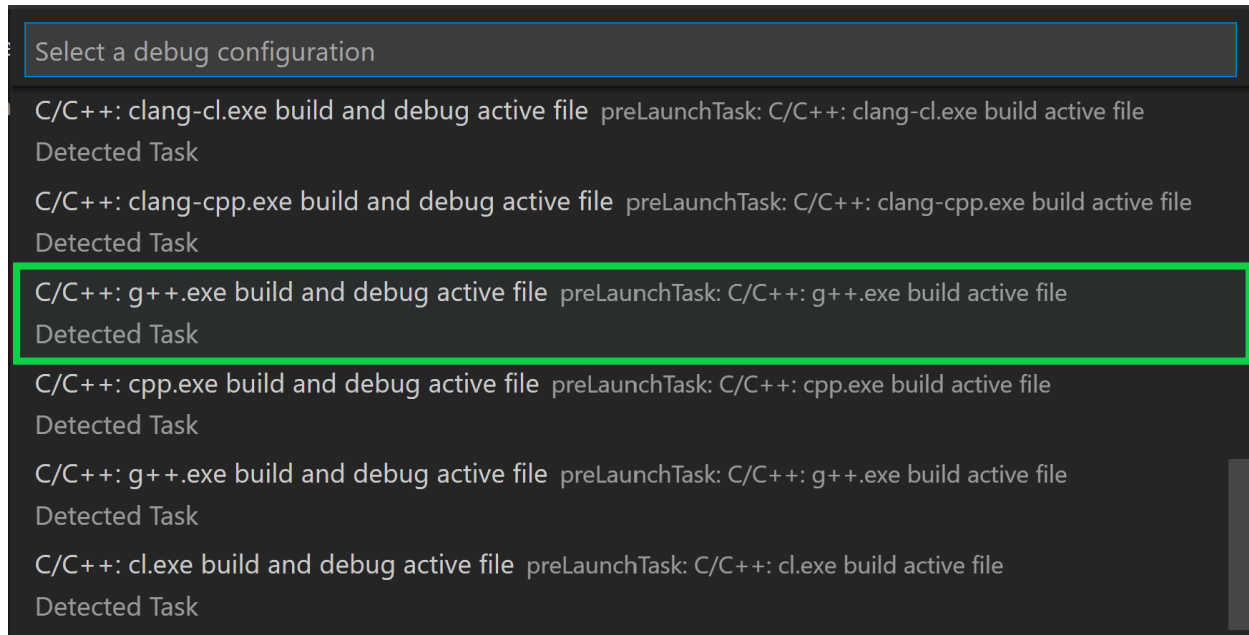
3. В раскрывающемся списке рядом с кнопкой воспроизведения выберите **Файл отладки C/C++**.



```
helloworld.cpp X {} tasks.json
helloworld.cpp > main()
6
7  int main()
8  {
9      vector<string> msg{"Hello", "C++", "World", "from", "VS Code", "and the C++ ext
10
11     for (const string &word : msg)
12     {
13         cout << word << " ";
14     }
15     cout << endl;
16 }
```

4. Выберите **C/C++: g++ для сборки и отладки активного файла** из списка обнаруженных компиляторов в вашей системе (вам будет предложено выбрать компилятор только при

первом запуске или отладке `helloworld.cpp`).

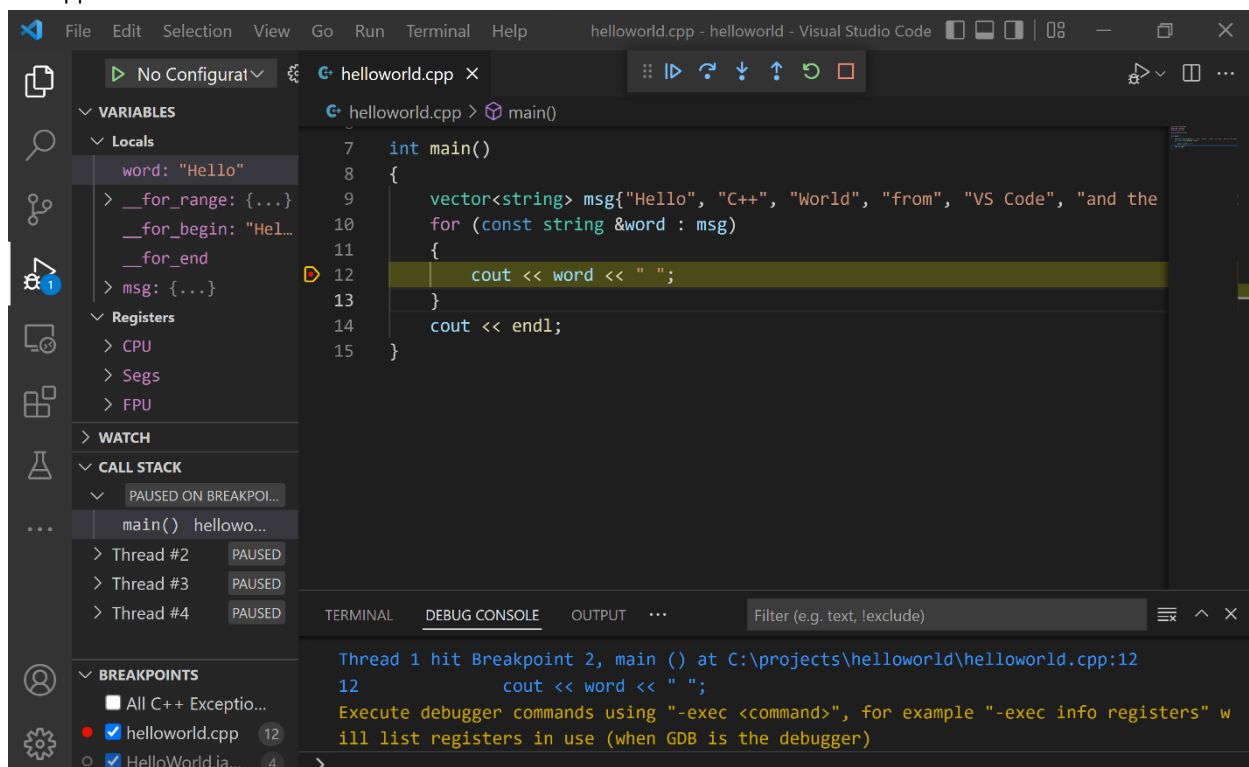


Кнопка воспроизведения имеет два режима: **запуск файла C/C++** и **отладка файла C/C++**. По умолчанию используется последний использованный режим. Если вы видите значок отладки на кнопке воспроизведения, вы можете просто нажать кнопку воспроизведения для отладки, а не использовать раскрывающийся список.

Изучение отладчика

Прежде чем вы начнёте изучать код, давайте обратим внимание на несколько изменений в пользовательском интерфейсе:

- Встроенный терминал отображается в нижней части редактора исходного кода. На вкладке **«Консоль отладки»** вы видите вывод, указывающий на то, что отладчик запущен.
- Редактор выделяет строку, в которой вы установили точку останова перед запуском отладчика:



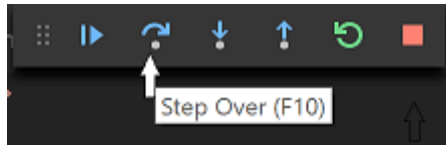
- В разделе «**Выполнить и отладить**» слева отображается информация об отладке. Позже в этом руководстве вы увидите пример.
- В верхней части редактора кода появляется панель управления отладкой. Вы можете перемещать её по экрану, удерживая точки с левой стороны.



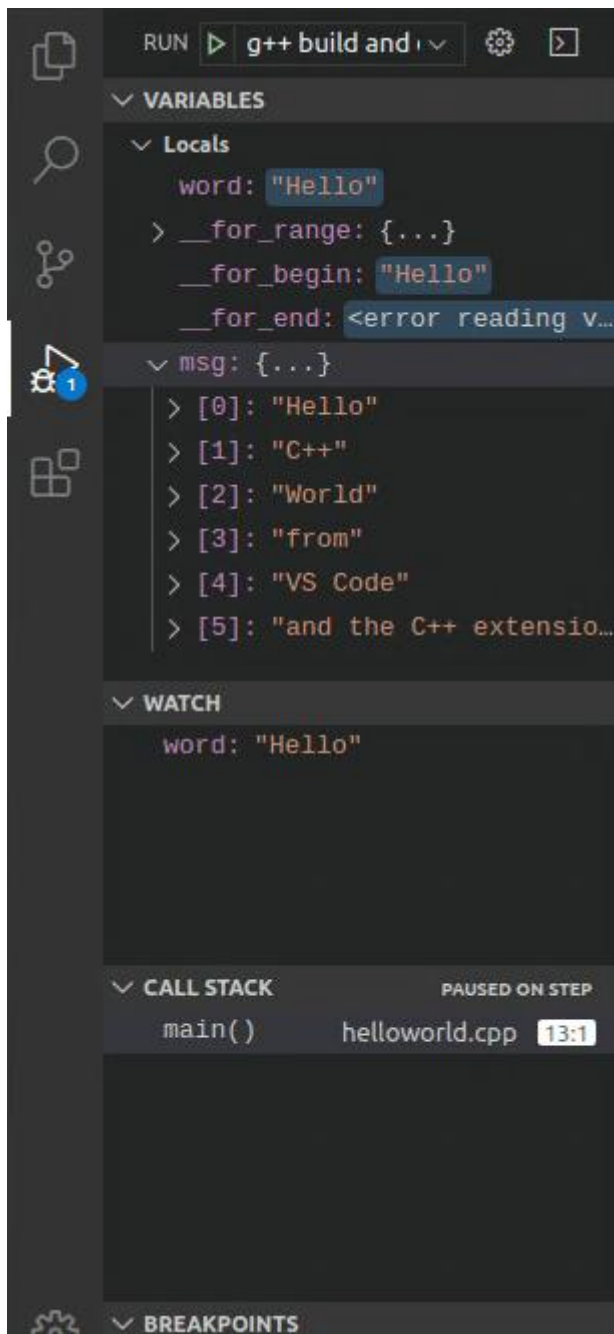
Пошаговый переход по коду

Теперь вы готовы приступить к пошаговому выполнению кода.

1. Выберите значок **Step over** на панели управления отладкой.



Это приведёт к переходу выполнения программы к первой строке цикла `for` и пропустит все внутренние вызовы функций в классах `vector` и `string`, которые вызываются при создании и инициализации переменной `msg`. Обратите внимание на изменения в окне **Переменные** слева.

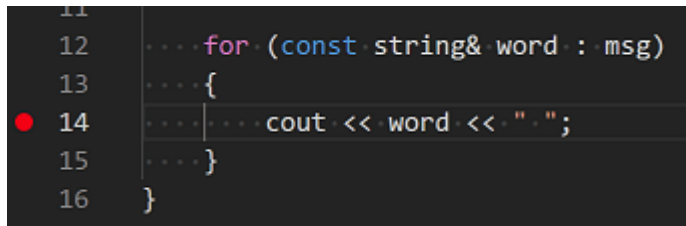


В этом случае ошибки ожидаемы, потому что, хотя имена переменных цикла теперь видны отладчику, оператор ещё не выполнен, поэтому в этот момент нечего читать. Однако содержимое `msg` видно, потому что оператор выполнен.

1. Нажмите **«Перейти»** ещё раз, чтобы перейти к следующему оператору в этой программе (пропуская весь внутренний код, который выполняется для инициализации цикла). Теперь в окне **«Переменные»** отображается информация о переменных цикла.
2. Нажмите **Step over** еще раз, чтобы выполнить `cout` инструкцию. (Обратите внимание, что расширение C++ не выводит никаких выходных данных на **консоль отладки** до завершения цикла.)
3. Если хотите, можете продолжать нажимать **«Шаг вперед»**, пока все слова в векторе не будут выведены на консоль. Но если вам интересно, попробуйте нажать кнопку **«Шаг в»**, чтобы пройти по исходному коду стандартной библиотеки C++!

Чтобы вернуться к своему коду, можно продолжать нажимать **«Шаг вперед»**. Другой способ — установить точку останова в коде, переключившись на вкладку `helloworld.cpp` в редакторе

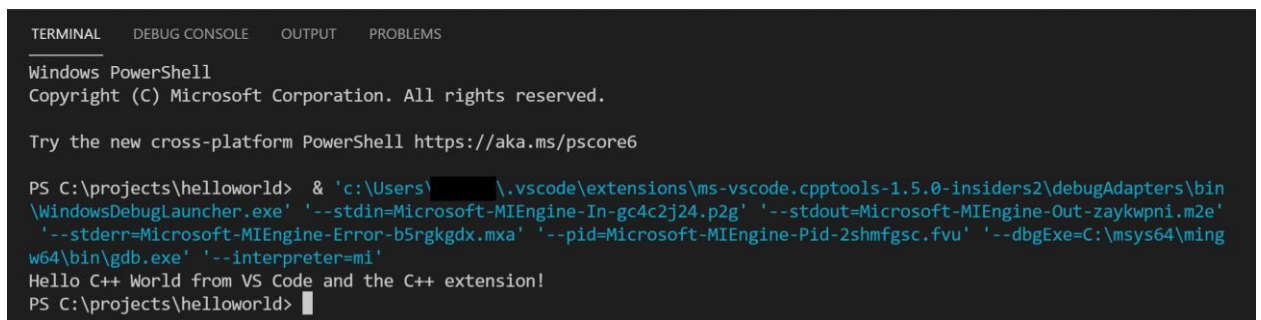
кода, поставив точку вставки в оператор `cout` внутри цикла и нажав F9. В левом столбце появится красная точка, указывающая на то, что на этой строке установлена точка останова.



```
11
12     ...for (const string& word : msg)
13     ...{
14     ...    cout << word << " ";
15     ...}
16 }
```

Затем нажмите F5, чтобы запустить выполнение с текущей строки в заголовке стандартной библиотеки. Выполнение прервётся на `cout`. Если хотите, можете снова нажать F9, чтобы отключить точку останова.

Когда цикл завершится, вы увидите результат в интегрированном терминале, а также другую диагностическую информацию, выводимую GDB.



```
TERMINAL  DEBUG CONSOLE  OUTPUT  PROBLEMS
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

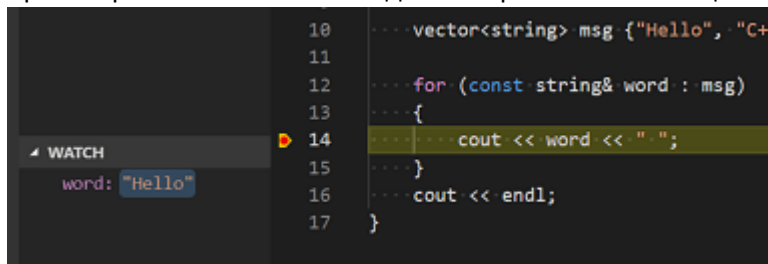
Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\projects\helloworld> & 'c:\Users\...\.vscode\extensions\ms-vscode.cpptools-1.5.0-insiders2\debugAdapters\bin\
\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-gc4c2j24.p2g' '--stdout=Microsoft-MIEngine-Out-zaykwpni.m2e'
'--stderr=Microsoft-MIEngine-Error-b5rgkgdx.mxa' '--pid=Microsoft-MIEngine-Pid-2shmfqsc.fvu' '--dbgExe=C:\msys64\ming
w64\bin\gdb.exe' '--interpreter=mi'
Hello C++ World from VS Code and the C++ extension!
PS C:\projects\helloworld> |
```

Отслеживание значений переменных

Иногда вам может понадобиться отслеживать значение переменной во время выполнения программы. Это можно сделать, установив **наблюдение** за переменной.

1. Поместите точку вставки внутрь цикла. В окне «**Наблюдение**» выберите знак «плюс» и в текстовом поле введите `word`, что является именем переменной цикла. Теперь просматривайте окно «Наблюдение» при выполнении цикла.



```
10     ...vector<string> msg {"Hello", "C++
11
12     ...for (const string& word : msg)
13     ...{
14     ...    cout << word << " ";
15     ...}
16     ...cout << endl;
17 }
```

WATCH

word: "Hello"

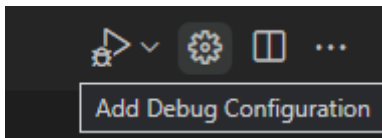
2. Добавьте ещё один цикл, добавив этот оператор перед циклом: `int i = 0;`. Затем внутри цикла добавьте этот оператор: `++i;`. Теперь добавьте цикл для `i` так же, как на предыдущем шаге.
3. Чтобы быстро просмотреть значение любой переменной во время приостановки выполнения на точке останова, наведите на неё указатель мыши.

Настройте отладку с помощью `launch.json`

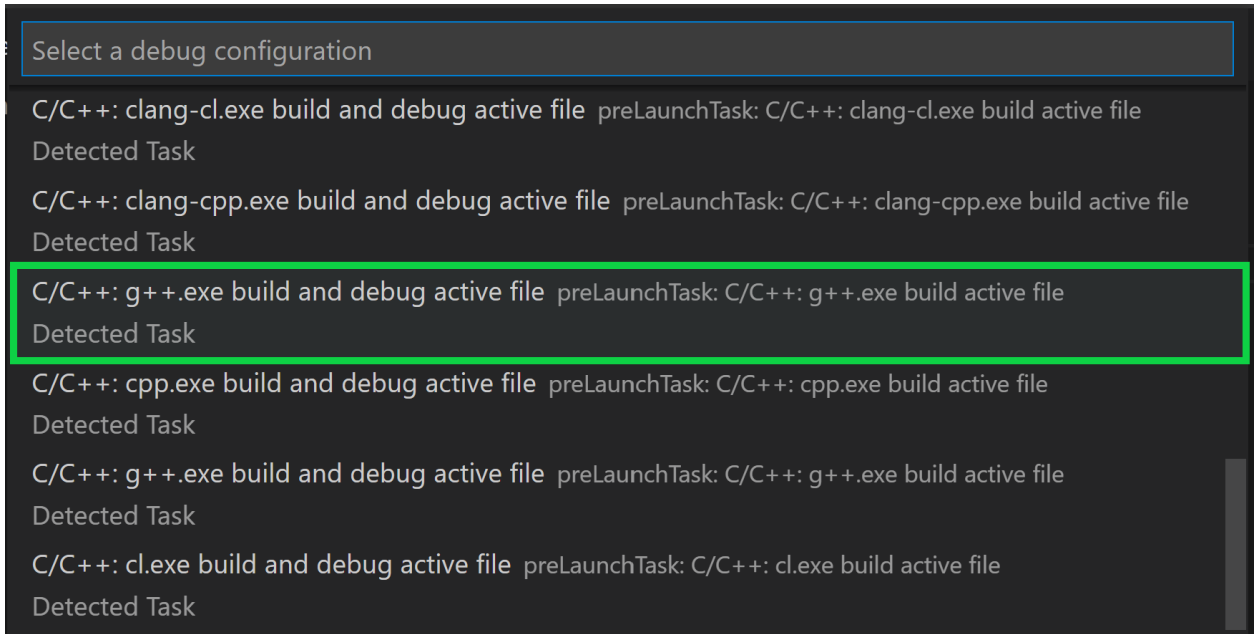
При отладке с помощью кнопки воспроизведения или F5 расширение C++ создаёт динамическую конфигурацию отладки на лету.

В некоторых случаях вам может понадобиться настроить конфигурацию отладки, например, указав аргументы, которые будут передаваться программе во время выполнения. Вы можете определить пользовательские конфигурации отладки в `launch.json` файле.

Чтобы создать `launch.json`, выберите **Добавить конфигурацию отладки** в раскрывающемся меню кнопки воспроизведения.



Затем вы увидите выпадающее меню с различными предустановленными конфигурациями отладки. Выберите **C/C++: g++.exe для сборки и отладки активного файла**.



VS Code создаёт файл `launch.json` в папке `.vscode`, который выглядит примерно так:

```
{
  "configurations": [
    {
      "name": "C/C++: g++.exe build and debug active file",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "gdb",
      "miDebuggerPath": "C:\\msys64\\ucrt64\\bin\\gdb.exe",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        },
        {
          "description": "Set Disassembly Flavor to Intel",
          "text": "-gdb-set disassembly-flavor intel",
          "ignoreFailures": true
        }
      ]
    },
    {
      "name": "C/C++: cl.exe build and debug active file",
      "type": "lldb",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "lldb",
      "miDebuggerPath": "C:\\Program Files\\LLDB\\bin\\lldb.exe",
      "setupCommands": [
        {
          "description": "Set Disassembly Flavor to Intel",
          "text": "-lldb-set disassembly-flavor intel",
          "ignoreFailures": true
        }
      ]
    }
  ],
  "preLaunchTask": "C/C++: g++.exe build active file"
}
```

```
],  
  "version": "2.0.0"  
}
```

В приведённом выше JSON `program` указывает на программу, которую вы хотите отладить. Здесь задана активная папка с файлами (`${fileDirname}`) и активное имя файла с расширением `.exe` (`${fileBasenameNoExtension}.exe`), которое, если `helloworld.cpp` является активным файлом, будет `helloworld.exe`. Свойство `args` — это массив аргументов, которые передаются программе во время выполнения.

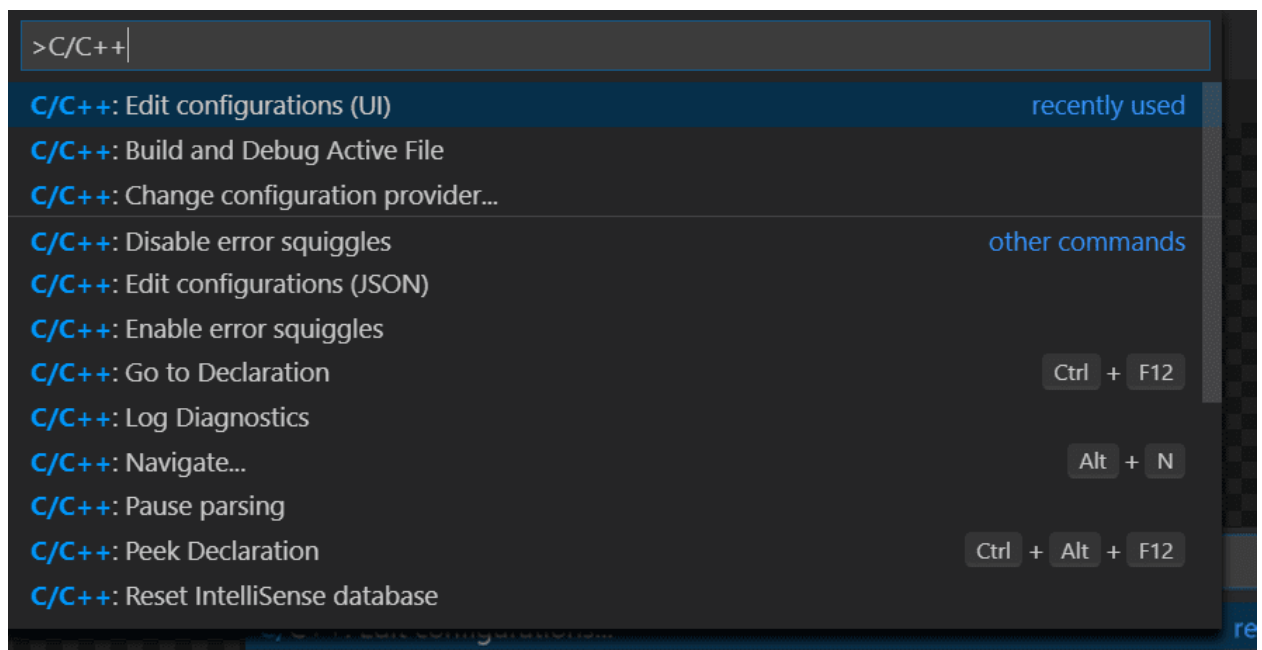
По умолчанию расширение C++ не добавляет никаких точек останова в ваш исходный код, а значение `stopAtEntry` равно `false`.

Измените значение `stopAtEntry` на `true`, чтобы отладчик останавливался на методе `main` при запуске отладки.

Добавление дополнительных настроек C / C ++

Если вам нужно больше возможностей для управления расширением C/C++, вы можете создать файл `c_cpp_properties.json` для изменения таких параметров, как путь к компилятору, пути включения, стандарт C++ (по умолчанию используется C++17) и многое другое.

Вы можете просмотреть пользовательский интерфейс конфигурации C/C++ с помощью команды **C/C++: Изменить конфигурации (UI)** в командной панели (`Ctrl+Shift+P`).



Откроется страница «Конфигурации C/C++». Когда вы вносите здесь изменения, VS Code записывает их в файл с именем `c_cpp_properties.json` в папке `.vscode`.

Здесь мы изменили **название конфигурации на GCC**, в раскрывающемся списке **путь компилятора** выбрали компилятор `g++` и **режим IntelliSense**, соответствующий компилятору (`gcc-x64`).

IntelliSense Configurations

Use this editor to edit IntelliSense settings defined in the underlying `c_cpp_properties.json` file. Changes made in this editor only apply to the selected configuration. To edit multiple configurations at once go to `c_cpp_properties.json`.

Configuration name

A friendly name that identifies a configuration. `Linux`, `Mac`, and `Win32` are special identifiers for configurations that will be auto-selected on those platforms.

Select a configuration set to edit.

GCC

Add Configuration

Compiler path

The full path to the compiler you use to build your project, e.g. `/usr/bin/gcc`, to enable more accurate IntelliSense. The extension will query the compiler to determine the system include paths and default defines to use for IntelliSense.

Specify a compiler path or select a detected compiler path from the drop-down list.

C:/Program Files/mingw-w64/x86_64-8.1.0-posix-seh-rt_v6-rev0/mingw64/bin/g++.exe

Compiler arguments

Compiler arguments to modify the includes or defines used, e.g. `-nostdinc++`, `-m32`, etc.

One argument per line.

IntelliSense mode

The IntelliSense mode to use that maps to an architecture-specific variant of MSVC, gcc, or Clang. If not set or if set to `${default}`, the extension will choose the default for that platform. Windows defaults to `msvc-x64`, Linux defaults to `gcc-x64`, and macOS defaults to `clang-x64`. Select a specific IntelliSense mode to override the `${default}` mode.

gcc-x64

Visual Studio Code сохраняет эти настройки в `.vscode\c_cpp_properties.json`. Если вы откроете этот файл напрямую, он должен выглядеть примерно так:

```
{
  "configurations": [
    {
      "name": "GCC",
      "includePath": ["${workspaceFolder}/**"],
      "defines": ["_DEBUG", "UNICODE", "_UNICODE"],
      "windowsSdkVersion": "10.0.22000.0",
      "compilerPath": "C:/msys64/mingw64/bin/g++.exe",
      "cStandard": "c17",
      "cppStandard": "c++17",
      "intelliSenseMode": "windows-gcc-x64"
    }
  ],
  "version": 4
}
```

Вам нужно добавить в массив **includePath** только в том случае, если ваша программа включает заголовочные файлы, которых нет в вашей рабочей среде или в пути к стандартной библиотеке. Настоятельно рекомендуется не добавлять системный путь включения в `includePath` настройку для компиляторов, которые мы поддерживаем.

Путь к компилятору

Расширение использует параметр `compilerPath` для определения пути к заголовочным файлам стандартной библиотеки C++. Когда расширение знает, где найти эти файлы, оно может предоставлять такие функции, как интеллектуальное завершение и **переход к определению**.

Расширение C/C++ пытается выбрать `compilerPath` компилятор по умолчанию на основе того, что оно находит в вашей системе. Расширение ищет компилятор в нескольких стандартных местах, но автоматически выбирает только тот, который находится в одной из папок «Program Files» или путь к которому указан в переменной среды PATH. Если можно найти компилятор Microsoft Visual C++, он будет выбран, в противном случае будет выбрана версия gcc, g++ или clang.

Если у вас установлено несколько компиляторов, вам может потребоваться изменить `compilerPath` в соответствии с предпочтительным компилятором для вашего проекта. Вы также можете использовать команду **C/C++: Выберите конфигурацию IntelliSense...** в командной панели, чтобы выбрать один из компиляторов, обнаруженных расширением.

Использование CMake

CMake — это кроссплатформенный инструмент с открытым исходным кодом, который использует независимые от компилятора и платформы файлы конфигурации для создания собственных файлов инструментов сборки, соответствующих вашему компилятору и платформе.

Расширение CMake Tools объединяет Visual Studio Code и CMake, упрощая настройку, сборку и отладку проекта на C++.

В этом руководстве вы будете использовать расширение CMake Tools для Visual Studio Code, чтобы настроить, собрать и отладить простой проект CMake на C++.

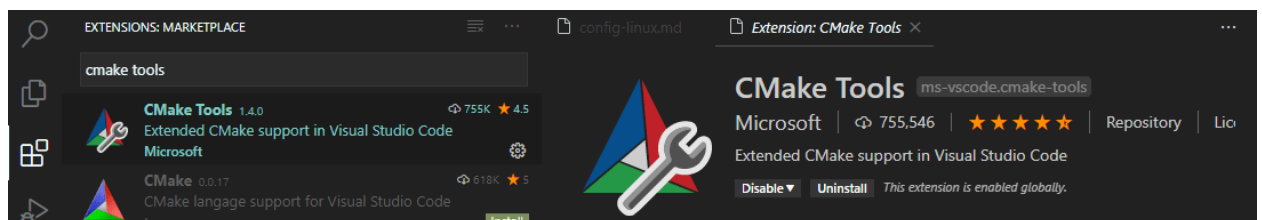
Установка CMake

Производится выбором соответствующего установщика: <https://cmake.org/download/>

Если установка производилась с помощью «инсталлера», то необходимо установить переключатель для автоматического прописывания пути к CMake в Path.

Установка расширения для работы с CMake

[Расширение CMake Tools для VS Code](#). Установите расширение CMake Tools, выполнив поиск «CMake tools» в разделе **Расширения** (Ctrl+Shift+X).



Убедитесь, что CMake установлен

Расширение VS Code CMake Tools работает с помощью CMake, установленного в вашей системе. Для достижения наилучших результатов используйте CMake версии 3.27 или выше.

Проверьте, установлен ли CMake в вашей системе. Откройте окно терминала и введите следующую команду:

```
cmake --version
```

Если CMake установлен, но команда проверки версии не работала, то необходимо вручную прописать путь к CMake в переменную окружения Path:

1. В строке поиска Windows введите **«Настройки»**, чтобы открыть «Настройки Windows».
2. Выполните поиск для **редактирования переменных среды для вашей учетной записи**.
3. В разделе **«Пользовательские переменные»** выберите переменную Path и нажмите **«Изменить»**.
4. Выберите **«Создать»** и добавьте в список папку назначения MinGW-w64, которую вы записали в процессе установки. Если вы использовали настройки по умолчанию, то это будет путь: C:\Program Files\CMake\bin.
5. Нажмите **ОК**, а затем снова нажмите **ОК** в окне **Переменные среды**, чтобы обновить PATH переменную среды. Вам необходимо повторно открыть все окна консоли, чтобы обновленная PATH переменная окружения была доступна.

Чтобы установить CMake или получить более позднюю версию, если у вас нет хотя бы версии 3.27, ознакомьтесь с инструкциями для вашей платформы в [репозитории Kitware APT](#). Установите версию 3.27 или более позднюю.

Создание проекта CMake hello world с помощью CMake Quick Start

В окне терминала создайте пустую папку с именем HelloWorld, перейдите в неё и откройте VS Code в этой папке, введя следующие команды:

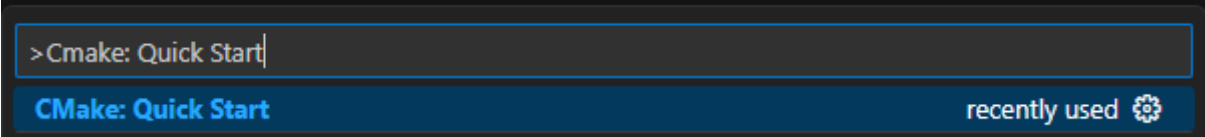
```
mkdir helloworld
cd helloworld
code .
```

Команда code . открывает VS Code в текущей рабочей папке, которая становится вашим «рабочим пространством».

Создание файла CMakeLists.txt

Расширение CMake Tools может создать для вас файлы для базового проекта CMake.

1. Откройте командную строку (Ctrl+Shift+P) и выполните команду CMake: Быстрый запуск:

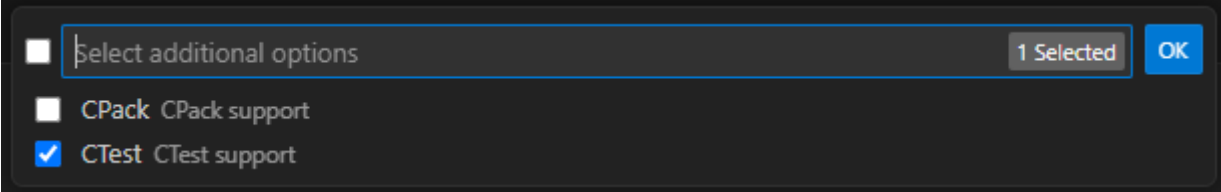


>Cmake: Quick Start|

CMake: Quick Start

recently used ⚙

2. Введите название проекта и выберите **C++** в качестве языка проекта. Эта информация будет записана в CMakeLists.txt и несколько исходных файлов.
Примечание: если в этой папке есть другие файлы исходного кода, которые вы хотите добавить в качестве целей в CmakeLists.txt, то теперь у вас будет возможность добавить их. Но в этом руководстве мы будем использовать только файл «Hello World».
3. Выберите **CTest** в качестве дополнительной опции, чтобы добавить поддержку тестирования. Вы также можете выбрать **CPack** для поддержки CPack.



☐ Select additional options

1 Selected

OK

☐ CPack CPack support

☒ CTest CTest support

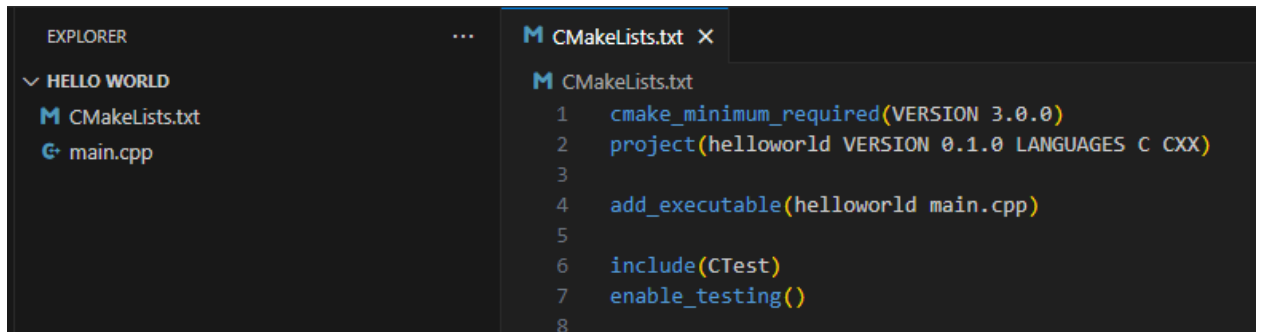
4. Затем выберите Исполняемый файл в качестве типа проекта, чтобы создать базовый исходный файл (main.cpp), включающий базовую функцию main().

Library Create a library

Executable Create an executable

Примечание: если бы вы хотели создать базовый исходный файл и файл заголовка, вы бы выбрали «Библиотеку». Но для этого урока подойдёт «Исполняемый файл». Если вам будет предложено настроить IntelliSense для папки, выберите «Разрешить».

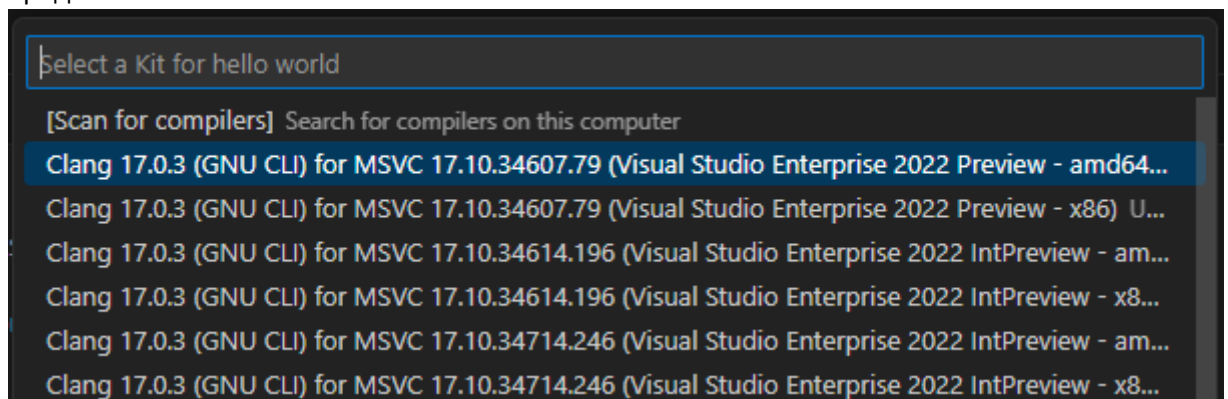
Это позволяет успешно создать файл CMakeLists.txt, который сообщает инструментам CMake, как собрать ваш проект.



Создание файла CMakePresets.json

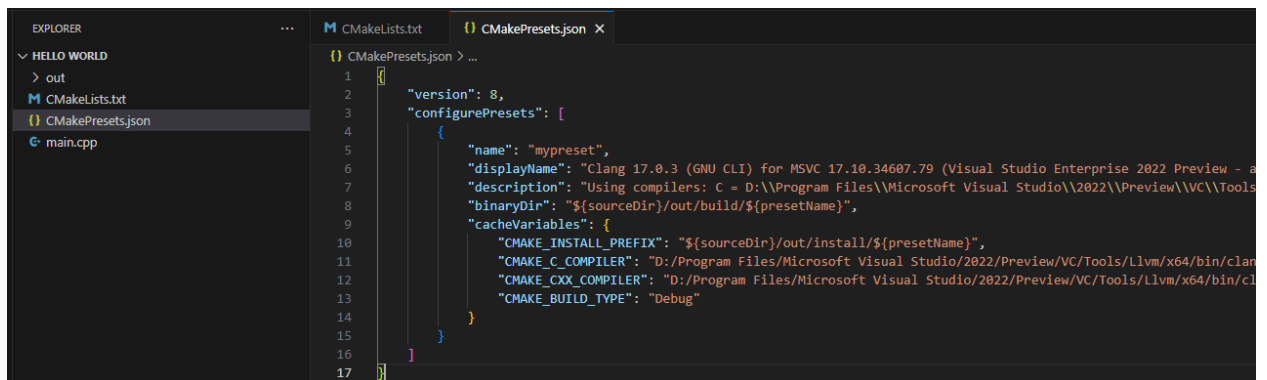
Затем перейдите к быстрому запуску CMake, чтобы создать CMakePresets.json файл.

1. Выберите Добавить новую предустановку и Создать с помощью компиляторов. Расширение автоматически сканирует наборы инструментов на вашем компьютере и создаёт список компиляторов, найденных в вашей системе.
2. Выберите компилятор, который вы хотите использовать. Например, в зависимости от установленных у вас компиляторов вы можете увидеть что-то вроде этого:



3. Введите имя для этого нового пресета. Название пресета будет записано в CMakePresets.json.

После выполнения этих шагов у вас должен получиться полноценный проект CMake hello world, содержащий следующие файлы: main.cpp, CMakeLists.txt, и CMakePresets.json.



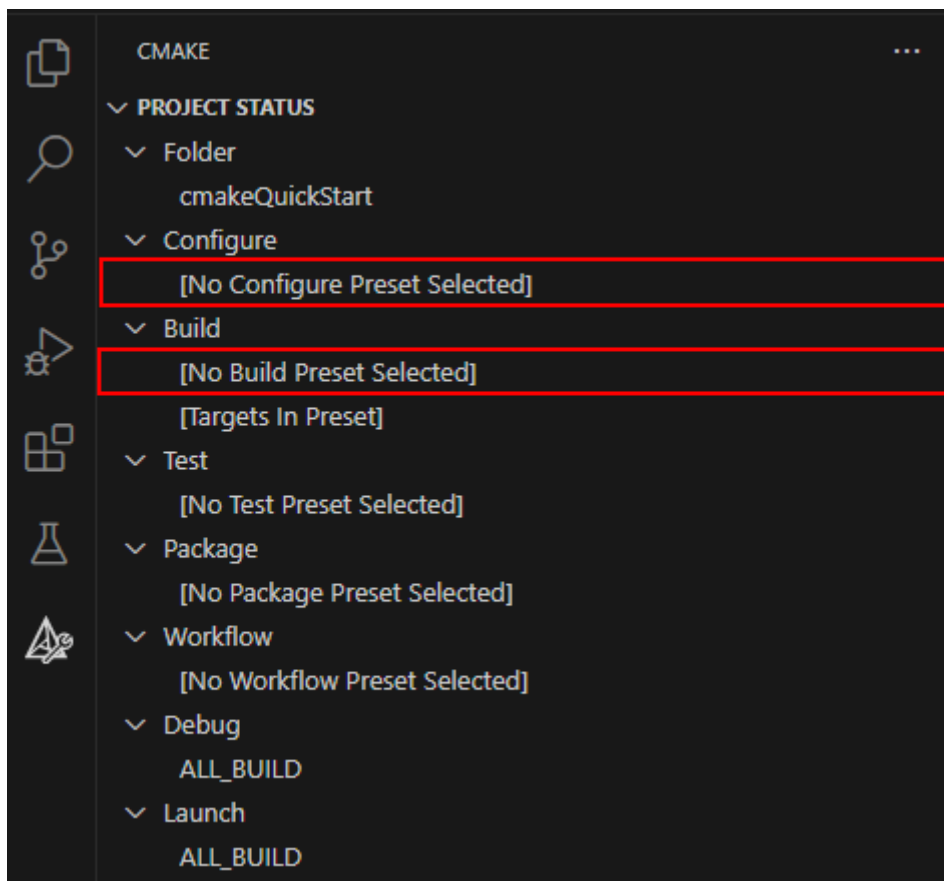
Настройка CMake с помощью предустановок

Рекомендуем использовать предустановки CMake для управления конфигурациями CMake. Предустановки CMake позволяют указать общий файл JSON, в котором хранятся все конфигурации для вашего проекта. Затем вы можете поделиться этим файлом с другими пользователями в разных IDE и операционных системах.

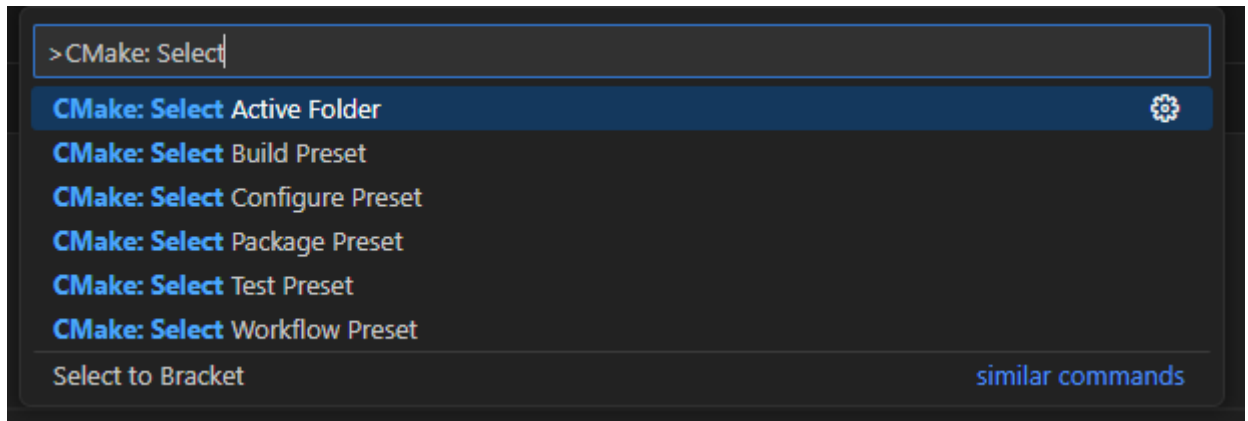
Если вы создали проект, следуя инструкциям в разделе «Создание проекта CMake», ваш проект настроен на использование предустановок CMake.

Если в вашем проекте есть файл `CMakePresets.json`, вы можете использовать предустановки «Настроить и собрать», чтобы указать, как собирать проект на вашем компьютере.

Вы можете просмотреть активную конфигурацию предустановок в разделе «Состояние проекта» в представлении «Инструменты CMake» в разделе «**Настройка**» и «**Сборка**». Вы можете в любой момент выбрать эти разделы, чтобы установить или изменить предустановки «Настройка» и «Сборка».



Вы также можете настроить любой из своих пресетов, выполнив команды **CMake: Выберите пресет для настройки** или **CMake: Выберите пресет для сборки** в палитре команд (Ctrl+Shift+P).

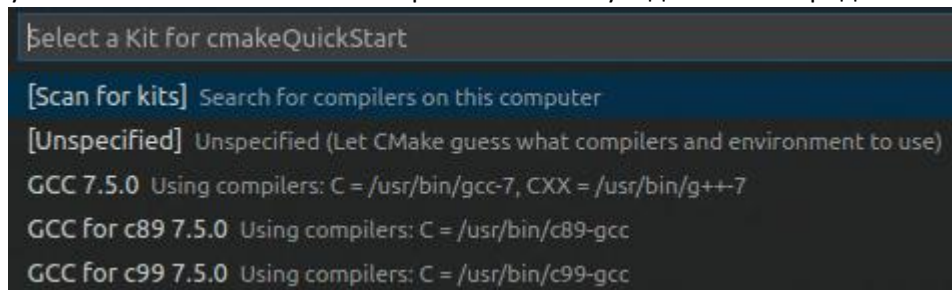


Настройка с помощью наборов CMake

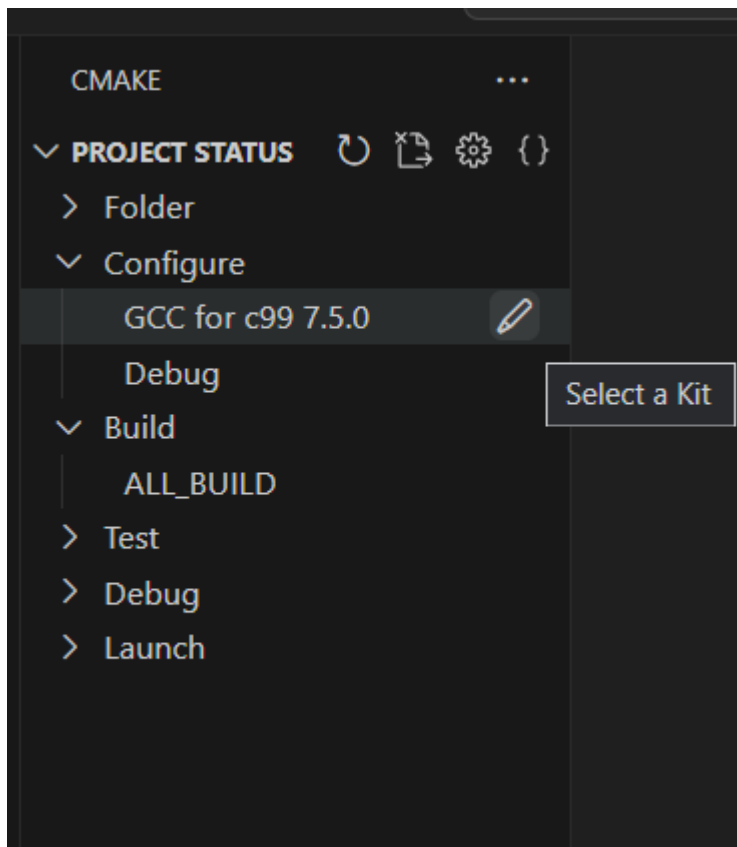
Если в вашем проекте нет файла `CMakePresets.json`, вам нужно использовать наборы инструментов. Набор инструментов — это компилятор, компоновщик и другие инструменты, используемые для сборки вашего проекта.

Сканировать наборы:

1. Откройте командную строку (Ctrl+Shift+P) и запустите **CMake: Выберите набор инструментов**. Расширение автоматически находит наборы инструментов на вашем компьютере и создаёт список компиляторов, доступных в вашей системе.
2. Выберите компилятор, который вы хотите использовать. Например, в зависимости от установленных вами компиляторов вы можете увидеть что-то вроде:



Набор, который вы выбрали ранее, теперь отображается в разделе **Состояние проекта** в представлении CMake Tools.



Чтобы изменить набор, вы можете выбрать его в разделе «**Состояние проекта**» в представлении «Инструменты CMake» или снова запустить команду «**CMake: выбрать набор**» в командной палитре. Если вы не видите нужный вам компилятор, вы можете отредактировать файл `cmake-tools-kits.json` в своем проекте. Чтобы отредактировать файл, откройте командную палитру (Ctrl+Shift+P) и запустите команду «**CMake: редактировать локальные наборы CMake**».

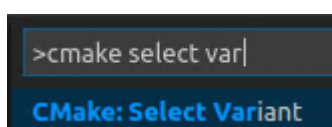
Затем вам нужно будет выбрать вариант.

Вариант содержит инструкции по сборке вашего проекта. По умолчанию расширение CMake Tools предоставляет четыре варианта, каждый из которых соответствует типу сборки по умолчанию: `Debug`, `Release`, `MinRelSize` и `RelWithDebInfo`. Эти параметры выполняют следующие действия:

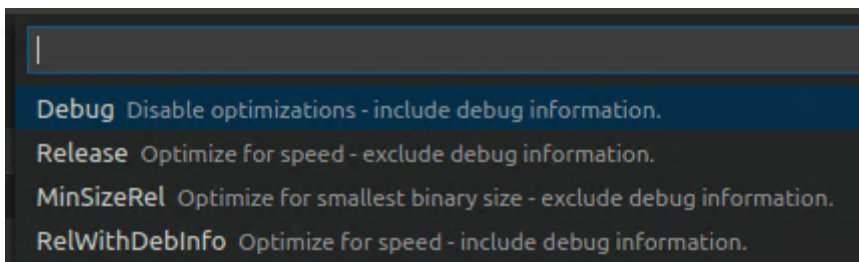
`Debug`: отключает оптимизацию и включает отладочную информацию. `Release`: включает оптимизацию, но не включает отладочную информацию. `MinRelSize`: оптимизирует размер. Отладочная информация отсутствует. `RelWithDebInfo`: оптимизирует скорость и включает отладочную информацию.

Чтобы выбрать вариант, откройте командную палитру (Ctrl+Shift+P) и выполните команду **CMake:**

Выберите вариант.



Выберите `Debug`, чтобы включить отладочную информацию в вашу сборку.



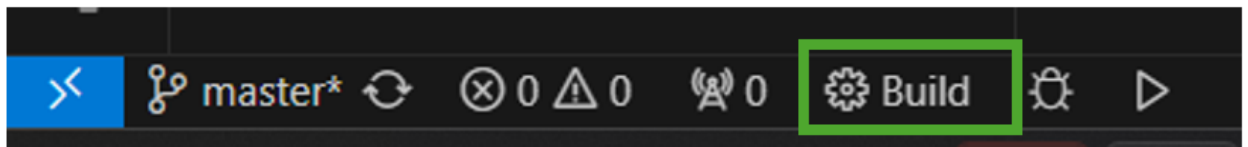
Выбранный вариант появится в строке состояния рядом с активным комплектом.

СMake: Настройка

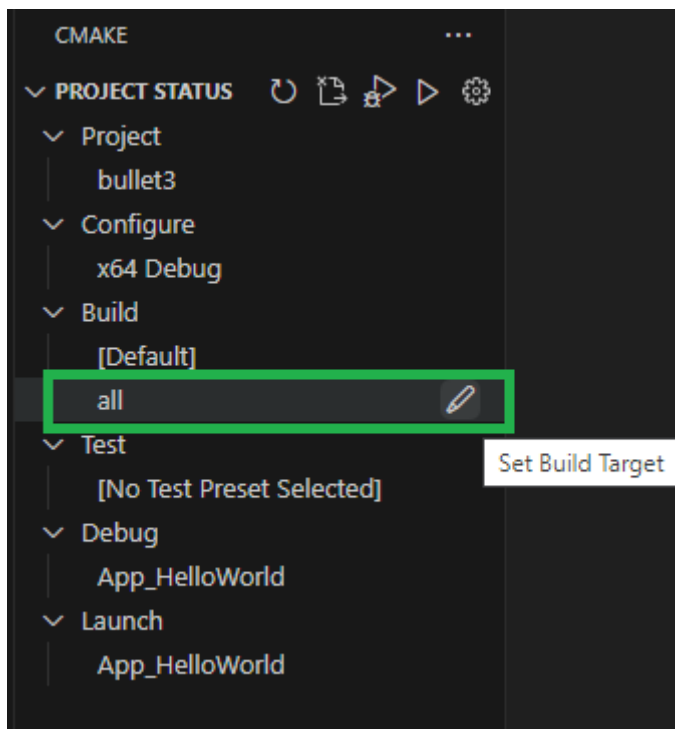
Теперь, когда вы выбрали настройки конфигурации с помощью предустановок или наборов/вариантов, откройте командную палитру (Ctrl+Shift+P) и выполните команду CMake: Configure для настройки вашего проекта. Это создаст файлы сборки в папке сборки проекта с использованием выбранных вами конфигураций.

Сборка приложения Hello World

После настройки проекта вы готовы к сборке. Откройте командную палитру (Ctrl+Shift+P) и выполните команду CMake: Build или нажмите кнопку Build в строке состояния.

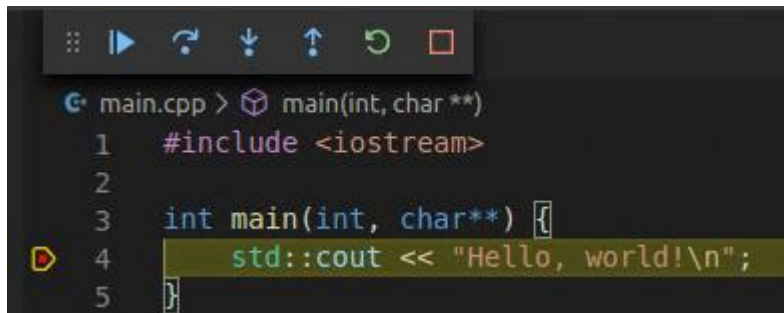


Вы можете выбрать, какие цели вы хотите создать, выбрав «СMake: установить цель сборки» в командной панели. По умолчанию CMake Tools создает все цели. Выбранная цель отобразится в разделе «Состояние проекта» на боковой панели CMake Tools под узлом «Сборка», и ее можно будет установить оттуда.



Отладка приложения Hello World

Чтобы запустить и отладить проект, откройте `main.cpp` и установите точку останова в строке `std::cout`. Затем откройте командную палитру (`Ctrl+Shift+P`) и запустите **CMake: Отладка**. Отладчик остановится в строке `std::cout`.



```
main.cpp > main(int, char **)
1  #include <iostream>
2
3  int main(int, char**) {
4      std::cout << "Hello, world!\\n";
5  }
```

Продолжайте и нажмите F5, чтобы продолжить.

Теперь вы использовали расширение VS Code CMake Tools для сборки и отладки приложения C++ на Ubuntu с помощью CMake. Шаги для других платформ аналогичны; разница заключается в том, как вы устанавливаете CMake и компилятор/отладчик для выбранной вами платформы.